

GCCB Workflow Guide

This document explains how to maintain the [Digital-Grinnell/collectionbuilder-csv](#) fork and create new Grinnell College CollectionBuilder (GCCB) sites.

Overview

This repository is a fork of [CollectionBuilder/collectionbuilder-csv](#) with Grinnell College-specific customizations and Oral History features integrated. It serves as the base template for all GCCB sites.

Maintaining the Fork: Syncing with Upstream

Initial Setup (One-Time)

If you haven't already configured the upstream remote, add it:

```
git remote add upstream
https://github.com/CollectionBuilder/collectionbuilder-csv.git
git remote -v # Verify remotes are configured
```

Regular Sync Process

1. Fetch upstream changes:

```
git fetch upstream
```

2. Switch to your main branch:

```
git checkout main
```

3. Merge upstream changes:

```
git merge upstream/main
```

4. Resolve conflicts (if any):

- Pay special attention to Grinnell-specific files:
 - [_config-OHD.yml](#)
 - [_sass/_ohd.scss](#)
 - [_includes/transcript/](#)

- `_data/config-transcript.csv`
- Custom files in `pages/` (e.g., `about-OHD.md`)
- Keep Grinnell customizations while incorporating upstream improvements
- Test thoroughly after resolving conflicts

5. Push updates to your fork:

```
git push origin main
```

Best Practices for Syncing

- **Sync regularly** (monthly or quarterly) to avoid large, complex merges
- **Document customizations** in comments to make conflict resolution easier
- **Test after merging** by building the site locally:

```
bundle exec jekyll serve
```

- **Create a backup branch** before major merges:

```
git checkout -b backup-pre-sync-$(date +%Y%m%d)
git push origin backup-pre-sync-$(date +%Y%m%d)
```

Creating New GCCB Sites

Method 1: Using GitHub Template (Recommended)

1. Create new repository from template:

- Go to <https://github.com/Digital-Grinnell/collectionbuilder-csv>
- Click "Use this template" → "Create a new repository"
- Name it following convention: `gccb-[collection-name]`
- Set visibility (usually private for college projects)

2. Clone to local machine:

```
git clone https://github.com/Digital-Grinnell/gccb-[collection-name].git
cd gccb-[collection-name]
```

3. Proceed to [Site Configuration](#)

Method 2: Manual Clone and Push

1. Clone this repository:

```
git clone https://github.com/Digital-Grinnell/collectionbuilder-csv.git gccb-[collection-name]
cd gccb-[collection-name]
```

2. Remove existing git history and create new repository:

```
rm -rf .git
git init
git add .
git commit -m "Initial commit from GCCB template"
```

3. Create new repository on GitHub (via web interface)

4. Push to new repository:

```
git remote add origin https://github.com/Digital-Grinnell/gccb-[collection-name].git
git branch -M main
git push -u origin main
```

Site Configuration

After creating a new GCCB site, customize it:

1. Update `_config.yml`

```
title: [Your Collection Title]
tagline: [Your Collection Tagline]
description: [Brief description]
author: Grinnell College Libraries

<h1 class="print-page-break">Collection-specific settings</h1>
url: https://[your-site-url]
baseurl: /[collection-name]

<h1 class="print-page-break">Metadata</h1>
metadata: [your-metadata-filename].csv
```

2. Prepare Collection Metadata

1. Create metadata CSV:

- Place in `_data/` directory
- Follow CollectionBuilder CSV schema (see [docs/metadata.md](#))
- Required fields: `objectid`, `title`, `format`, `filename`
- See `_data/demo-mixed.csv` for examples

2. Update configuration files in `_data/`:

- `config-metadata.csv` - Display fields for item pages
- `config-browse.csv` - Browse page configuration
- `config-search.csv` - Search fields
- `config-nav.csv` - Navigation menu
- `config-map.csv` - If using map feature
- `config-table.csv` - Table display settings

3. Add Digital Objects

1. Place files in `objects/` directory:

- Images: JPG, PNG
- PDFs: For documents
- Audio/Video: MP3, MP4 (or use external hosting)

2. For large files or hosted content:

- Use external URLs in `filename` field
- Set `object_location` in metadata

4. Customize Pages

Edit markdown files in `pages/`:

- `about.md` - Collection information
- `index.md` - Home page (if not using layout default)
- Add custom pages as needed

5. Oral History Features (Optional)

If including oral history content:

1. Enable transcript feature in `_config.yml`:

```
transcript: true
```

2. Add transcript files:

- Place in `_data/transcripts/`
- Name matching `objectid` from metadata
- Format: TXT, VTT, or JSON

3. Configure `_data/config-transcript.csv`

4. Use OHD-specific config if needed:

- Copy settings from `_config-OHD.yml`

6. Theme Customization

Customize appearance:

- `_data/theme.yml` - General theme settings
- `_data/config-theme-colors.csv` - Color scheme
- `_sass/_custom.scss` - Custom CSS

Building and Testing

Local Development

1. Install dependencies:

```
bundle install
```

2. Serve site locally:

```
bundle exec jekyll serve
```

3. View at: `http://localhost:4000`

Using Rake Tasks

The project includes rake tasks for common operations:

```
<h1 class="print-page-break">Generate derivatives for images</h1>
rake generate_derivatives

<h1 class="print-page-break">Deploy to server (if configured)</h1>
rake deploy

<h1 class="print-page-break">See all available tasks</h1>
rake -T
```

See [docs/rake_tasks.md](#) for details.

Deployment

GitHub Pages

1. Enable in repository settings:

- Settings → Pages
- Source: Deploy from branch
- Branch: `main`, folder: `/` or `/docs`

2. Update `_config.yml`:

```
url: https://digital-grinnell.github.io
baseurl: /gccb-[collection-name]
```

Custom Server

Configure deployment in [Rakefile](#) or use provided scripts in [raketlib/](#).

Maintaining GCCB Sites

Regular Updates

1. Pull updates from template (optional):

```
git remote add template https://github.com/Digital-Grinnell/collectionbuilder-csv.git
git fetch template
git merge template/main --allow-unrelated-histories
# Resolve conflicts, keeping site-specific content
```

2. Keep dependencies updated:

```
bundle update
```

3. Test after updates:

```
bundle exec jekyll serve
```

Content Updates

- **Metadata changes:** Edit CSV in `_data/`
- **New items:** Add to metadata CSV and `objects/` directory
- **Page edits:** Update markdown files in `pages/`
- Commit and push changes to deploy

Getting Help

- **CollectionBuilder Documentation:** <https://collectionbuilder.github.io/cb-docs/>
- **Local Documentation:** See `docs/` directory
- **Grinnell-Specific Features:** See `_config-OHD.yml` and oral history includes

Repository Structure Reference

```
Key Grinnell-Specific Files:
├── _config-OHD.yml           # Oral History configuration
├── _sass/_ohd.scss          # OHD-specific styles
├── _includes/transcript/    # Transcript display components
├── _data/config-transcript.csv # Transcript configuration
└── pages/about-OHD.md       # OHD about page example
```

Standard CollectionBuilder Structure:

— _data/	# Metadata and configuration CSVs
— _includes/	# Reusable components
— _layouts/	# Page layouts
— _plugins/	# Custom Jekyll plugins
— objects/	# Digital objects
— pages/	# Site pages
— _config.yml	# Main site configuration

Troubleshooting

Build Errors

- Verify Ruby and bundler versions match project requirements
- Try `bundle clean --force && bundle install`
- Check for syntax errors in YAML files

Merge Conflicts

- Review changes carefully in Grinnell-specific files
- Keep customizations while adopting upstream improvements
- Test thoroughly after resolving

Missing Objects

- Verify filenames in metadata match actual files in `objects/`
- Check file paths and extensions (case-sensitive on some systems)

Creating Azure Static Web Apps

This repository includes [scripts/create_azure_static_web_app.py](#) for provisioning a new Azure Static Web App from the current clone. It is intended for cases where you want to create a fresh deployment that follows the same Jekyll build pattern as the existing [victorious-sea-07a73ff10](#) site.

Requirements

- Azure CLI installed and authenticated with `az login`
- GitHub CLI installed and authenticated with `gh auth token`, or pass a token with `--token`
- A GitHub repository origin remote that points to the repo you want to deploy

Example

```
python3 scripts/create_azure_static_web_app.py \  
  --name my-static-web-app \  
  --resource-group my-static-web-app-rg
```

The script creates the resource group if needed, provisions the Azure Static Web App, and then prints the production address in the form [https://<defaultHostname>](#) when the deployment is ready.

Last Updated: June 2026

Maintained By: Grinnell College • Digital Library Applications Developer